PYTHON CONTROL FLOW

- Control flow defines the **sequence in which statements are executed** in a Python program.
- Normally, Python executes code **line by line from top to bottom**.
- Control flow statements allow the program to **change this order** based on conditions, loops, or jumps.

Control flow helps in:

- Making **decisions**
- **Repeating** tasks
- **Skipping or stopping** execution
- Writing **logical and efficient programs**

Categories of Control Flow in Python

Python control flow is divided into **three major categories**:

1. **Decision-Making Statements**
2. **Looping Statements**
3. **Jump / Control Transfer Statements**

---

1. Decision-Making Statements :

- Decision-making statements allow the program to **choose different paths** based on conditions.
- Conditions usually return **True or False**.

a) if Statement:

- The if statement checks a condition.
- If the condition is **true**, the block of code inside the if is executed.
- If the condition is **false**, the block is skipped.

Example code:

```python
age = 20
if age >= 18:
    print("You are eligible to vote")
```

b) if–else Statement :

- The if–else statement provides **two execution paths**.
- If the condition is true, if block executes.
- If the condition is false, else block executes.

Example code:

```python
num = 7
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

c) if–elif–else Statement :

- Used when **multiple conditions** need to be checked.
- Python checks conditions from **top to bottom**.

Example code:

```python
marks = 82
if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

d) Nested if Statement :

- An if statement inside another if statement is called nested if.

```python
age = 20
citizen = True

if age >= 18:
    if citizen:
        print("Eligible to vote")
```

---

2. Looping Statements :

Loops are used to **execute a block of code repeatedly** until a condition is met. They reduce code repetition and improve efficiency.

a) for Loop:

The for loop is used to iterate over **sequences** such as:

- List
- Tuple
- String
- Range

Iterating over a List:

- A list is an ordered collection of elements.
- The for loop accesses **each element one by one** from the list.

  Example code:

```python
numbers = [10, 20, 30, 40]

for num in numbers:
    print(num)
```

Iterating over a Tuple :

- A tuple is similar to a list but **immutable**.
- The for loop can read elements from a tuple but **cannot modify them**.

Example code:

```python
colors = ("red", "green", "blue")

for color in colors:
    print(color)
```

Iterating over a String :

- A string is a sequence of characters.
- The for loop iterates **character by character** in a string.

Example code:

```python
name = "Python"

for ch in name:
    print(ch)
```

Iterating using range() :

- The range() function generates a sequence of numbers.
- It is widely used in loops where a **fixed number of iterations** is required.

Example code:

```python
for i in range(1, 6):
    print(i)
```

b) while Loop :

- The while loop executes as long as the condition is **true**.
- It is used when the **number of iterations is unknown**.

Example code:

```python
i = 1
while i <= 5:
    print(i)
    i += 1
```

c) Nested Loops :

- A loop inside another loop is called a nested loop.
- Used in pattern printing and matrix operations.

Example code:

```python
for i in range(1, 4):
    for j in range(1, 4):
        print(i, j)
```

---

3 .Jump / Control Transfer Statements :

- Jump statements change the **normal flow of loop execution**.

a) break Statement

- The break statement is used to **immediately terminate a loop**.

- When Python encounters break, it **stops the loop execution** and control moves **outside the loop**.

  Example code:

```python
for i in range(1, 6):
    if i == 4:
        break
    print(i)
```

b) continue Statement :

- The continue statement **skips the current iteration** and moves to the **next iteration** of the loop.
- The loop does not stop; it just **ignores the remaining code in that iteration**.

  Example code:

```python
for i in range(1, 11):
    if i % 2 == 0:
        continue
    print(i)
```

c) pass Statement :

- The pass statement **does nothing**.
- It is used when a statement is required **syntactically**, but no action is needed yet.

  Example code:

```python
for i in range(5):
    if i == 2:
        pass
    print(i)
```

---

4. Match-Case Statement :

- The match-case statement is similar to **switch-case** in other programming languages.
- It allows Python to **compare a value against multiple patterns** and execute the matching block.

  Example code:

```python
choice = 2
match choice:
    case 1:
        print("Add")
    case 2:
        print("Delete")
    case 3:
        print("Update")
    case _:
        print("Invalid choice")
```

---

5. Control Flow with Functions :

Functions control program flow using:

- if-else conditions
- return statements

- Parameters and arguments

The return statement sends a value back and **stops function execution**.

Example code:

```python
def find_max(a, b):
    if a > b:
        return a
    else:
        return b

print(find_max(10, 20))
```